

8 Merge Sort

Code:-

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
void merge (vector<int> &arr, int left, int mid, int right) {
```

```
    int m1 = mid - left + 1;
```

```
    int m2 = right - mid;
```

```
    vector<int> L(m1), R(m2);
```

```
    for (int i = 0; i < m1; i++)
```

```
        L[i] = arr[left + i];
```

```
    for (int i = 0; i < m2; i++)
```

```
        R[i] = arr[mid + 1 + i];
```

```
    int i = 0, j = 0, k = left;
```

```
    while (i < m1 && j < m2) {
```

```
        if (L[i] <= R[j]) {
```

```
            arr[k++] = L[i++];
```

```
        } else {
```

```
            arr[k++] = R[j++];
```

```
        }
```

```
    while (i < m1)
```

```
        arr[k++] = L[i++];
```

```
while (j < m)
```

```
arr[k++] = R[j++];
```

```
}
```

```
void mergesort (vector<int> &arr, int left, int right)
```

```
{ if (left < right) {
```

```
    int mid = left + (right - left) / 2;
```

```
    mergesort (arr, left, mid);
```

```
    mergesort (arr, mid+1, right);
```

```
    merge (arr, left, mid, right);
```

```
}
```

```
}
```

```
int n;
```

```
cin >> n;
```

```
vector<int> arr(n);
```

```
for (int i = 0
```

#

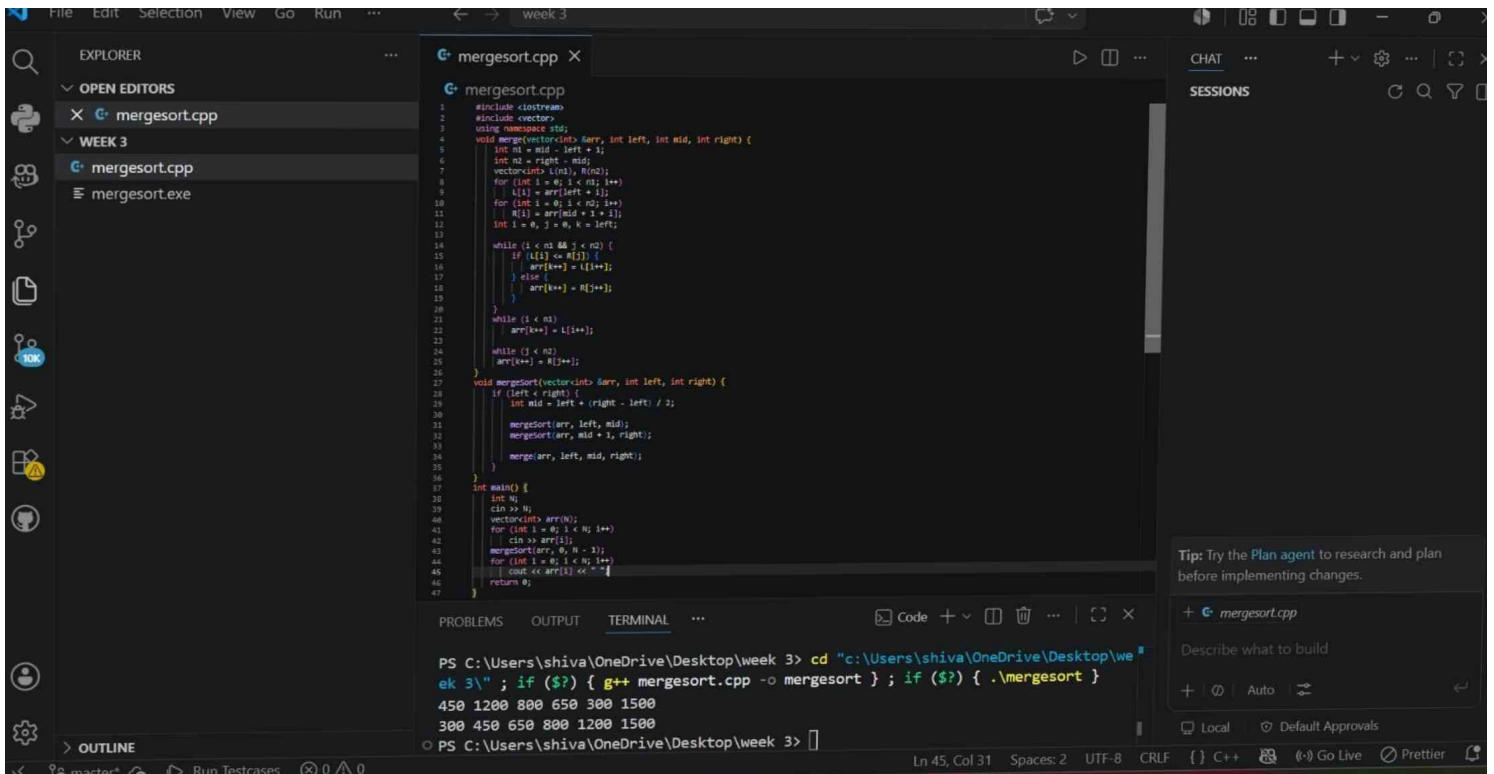
Input 6

6

~~1500 1200 800 650 450 300~~ 450 1200 800 650 300 1500

#Output

300 450 650 800 1200 1500



Q Merge Sort 2

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
void merge (vector<int> &arr, int left, int mid, int right)
```

```
{ int m1 = mid - left + 1;
```

```
int m2 = right - mid;
```

```
vector<int> L(m1), R(m2);
```

```
for (int i=0; i<m1; i++)
```

```
    L[i] = arr[left + i];
```

```
for (int j=0; j<m2; j++)
```

```
    R[j] = arr[mid + j + 1];
```

```
int i=0, j=0, k=left;
```

```
while (i<m1 && j<m2)
```

```
{ if (L[i] <= R[j])
```

```
    arr[k++] = L[i++];
```

```
else
```

```
    arr[k++] = R[j++];
```

```
}
```

```
while (i<m1)
```

```
    arr[k++] = L[i++];
```

```
while (j<m2)
```

```
{ arr[k++] = R[j++];
```

```
}
```



```
void mergesort (vector<int> &arr, int left, int right)
```

```
{ if (left < right)
```

```
{ int mid = left + (right - left) / 2;
```

```
mergesort(arr, left, mid);
```

```
mergesort(arr, mid + 1, right);
```

```
merge(arr, left, mid, right);
```

```
}
```

```
}
```

```
}
```

```
int main()
```

```
{ int N;
```

```
cin >> N;
```

```
vector<int> arr(N);
```

```
for (int i = 0; i < N; i++)
```

```
cin >> arr[i];
```

```
mergesort(arr, 0, N-1);
```

```
for (int i = 0; i < N; i++)
```

```
cout << arr[i] << " ";
```

```
cout << endl;
```

```
int median;
```

```
if (N % 2 == 1)
```

```
median = arr[N/2];
```

```
else
```

```

median = (arr[N/2 - 1] + arr[N/2]) / 2;
cout << "Median: " << median << endl;

int count = 0;
for (int x : arr) {
    if (x > median)
        count++;
}
cout << "Numbers Above Median: " << count << endl;
cout << "Difference: " << arr[N-1] - arr[0] << endl;

return 0;
}

```

#input
7

12 25 18 80 15 22 40

#output

Median: 22

~~Other~~
Numbers Above Median: 3

Difference: 28

```

median = (arr[N/2 - 1] + arr[N/2]) / 2;
cout << "Median: " << median << endl;

int count = 0;
for (int x: arr) {
    if (x > median)
        count++;
}

cout << "Orders Above Median: " << count << endl;
cout << "Difference: " << arr[N-1] - arr[0] << endl;

return 0;
}

```

#input

7

12 25 18 80 15 22 40

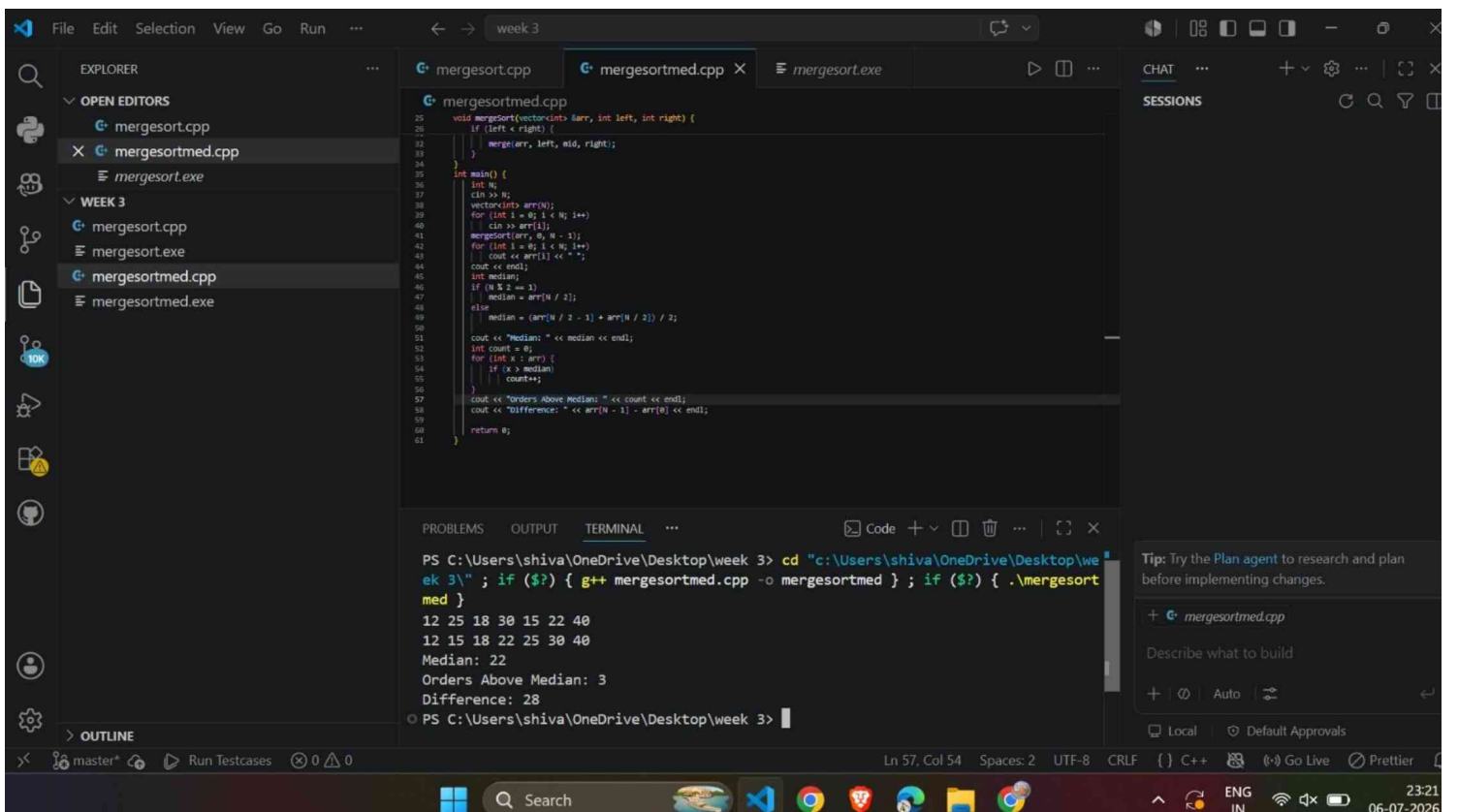
#output

Median: 22

~~Other~~

Orders Above Median: 3

Difference: 28



3) QuickSort

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
int partition (vector<int> &arr, int low, int high)
```

```
{  
    int pivot = arr[high];
```

```
    int i = low - 1;
```

```
    for (int j = low; j < high; j++)
```

```
        if (arr[j] <= pivot)
```

```
            i++;
```

```
        swap (arr[i], arr[j]);
```

```
}
```

```
}
```

```
    swap (arr[i+1], arr[high]);
```

```
    return i+1;
```

```
}
```

```
void quicksort (vector<int> &arr, int low, int high)
```

```
{  
    if (low < high)
```

```
        int pi = partition (arr, low, high);
```

```
        quicksort (arr, low, pi-1);
```

```
        quicksort (arr, pi+1, high);
```

```
}
```

int main() {

int N;

cin >> N;

vector<int> arr(N);

for (int i=0; i<N; i++)

cin >> arr[i];

quicksort(arr, 0, N-1);

for (int i=0; i<N; i++)

cout << arr[i] << " ";

return 0;

}

#input

6

120 45 78 200 60 95

#output

45 60 78 95 120 200.


```

4) #include <iostream>
#include <vector>
using namespace std;

int partition (vector <long long> &arr, int low, int high) {
    long long pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap (arr[i], arr[j]);
        }
    }
    swap (arr[i+1], arr[high]);
    return i+1;
}

void quickSort (vector <long long> &arr, int low, int high) {
    if (low < high) {
        int pi = partition (arr, low, high);
        quickSort (arr, low, pi-1);
        quickSort (arr, pi+1, high);
    }
}

```



```

int main() {
    int N;
    cin >> N;
    Vector<long long> arr(N);
    long long sum = 0;
    for (int i = 0; i < N; i++) {
        cin >> arr[i];
        sum += arr[i];
    }
    quickSort(arr, 0, N-1);
    for (int i = 0; i < N; i++)
        cout << arr[i] << " ";
    cout << endl;
    cout << "Top 5: ";

```

```

    long long topsum = 0;
    for (int i = 0; i < 5; i++) {
        cout << arr[i] << " ";
        topsum += arr[i];
    }

```

```

cout << endl;
cout << "Average of Top 5: " << topSum / 5 << endl;
double overallAvg = (double) sum / N;
int count = 0;
for (int i = 0; i < N; i++) {
    if (arr[i] > overallAvg)
        count++;
}
cout << "Values Above Overall Avg: " << count << endl;
return 0;
}

```

input

8

500 1200 700 2000 900 1500 3000 2500

output.

3000 2500 2000 1500 1200 900 700 500

Top 5: 3000 2500 2000 1500 1200

Average of Top 5: 2040

Values Above overall Average: 4



Heap Sort

```
#include <iostream>
#include <vector>
using namespace std;

void heapify(vector<int> &arr, int n, int i) {
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapsort(vector<int> &arr, int n) {
    for (int i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n-1; i > 0; i--) {

```



```

    swap(arr[i], arr[0]);
    heapify(arr, i, 0);
}
}
int main() {
    int N;
    cin >> N;
    vector<int> arr(N);
    for (int i=0; i<N; i++)
        cin >> arr[i];
    heapsort(arr, N);
    for (int i=0; i<N; i++)
        cout << arr[i] << " ";
    return 0;
}

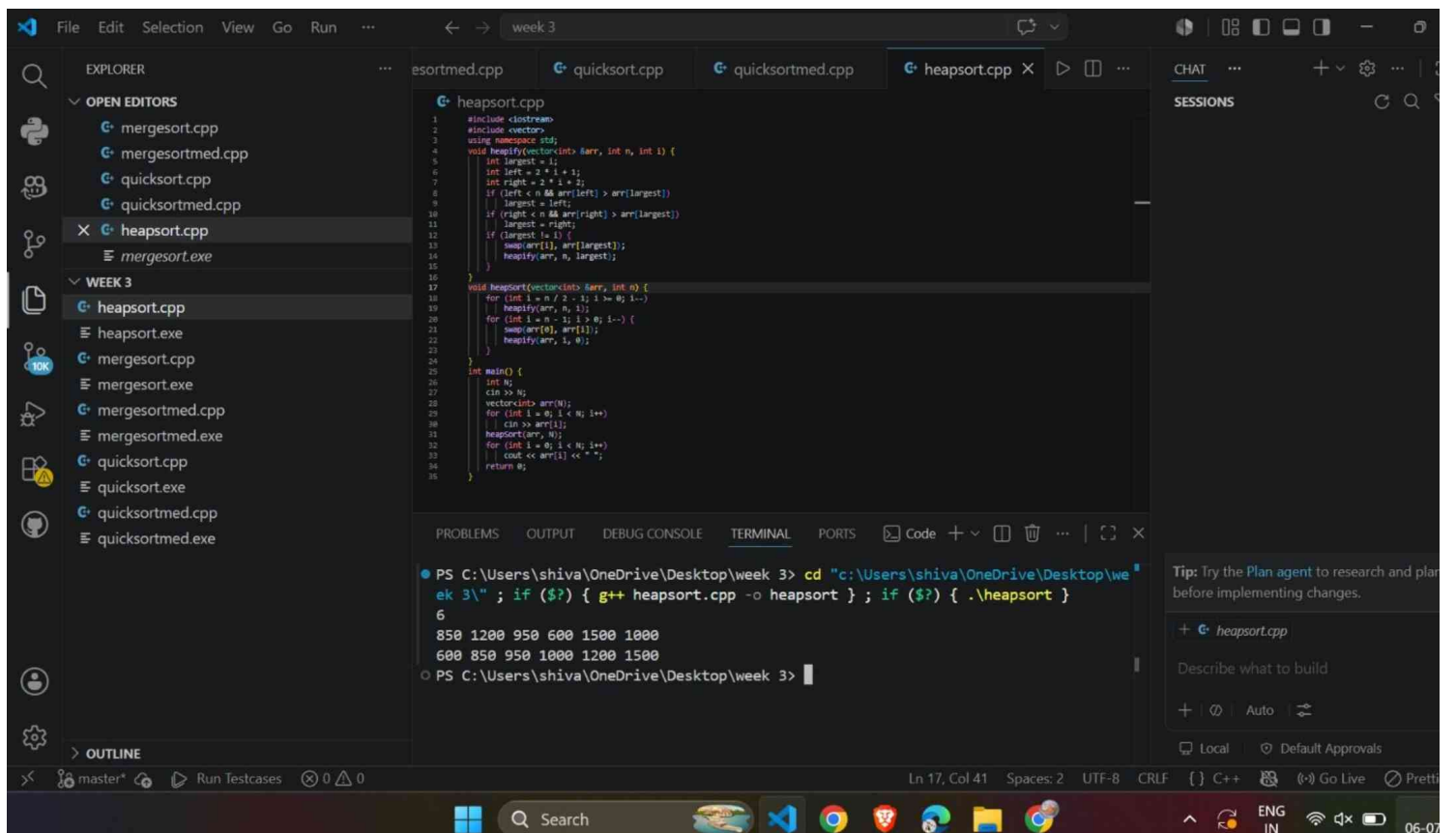
```

#input

6
856 1200 950 600 1500 1000

#output

600 856 950 1000 1200 1500



```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
int partition(vector<long long> &arr, int low, int high)
```

```
{  
    long long pivot = arr[high];
```

```
    int i = low - 1;
```

```
    for (int j = low; j < high; j++) {
```

```
        if (arr[j] >= pivot) {
```

```
            i++;
```

```
            swap(arr[j], arr[i]);
```

```
        }
```

```
    }
```

```
    swap(arr[i+1], arr[high]);
```

```
    return i+1;
```

```
}
```

```
void quicksort(vector<long long> &arr, int low, int  
    high) {
```

```
    if (low < high) {
```

```
        int pi = partition(arr, low, high);
```

```
        quicksort(arr, low, pi-1);
```

```
        quicksort(arr, pi+1, high);
```

```
    }
```

```
}
```

```
int main() {
```

```
int N;
```

```
cin >> N;
```

```
vector<long long> arr(N);
```

```
long long sum=0;
```

```
for (int i=0; i<N; i++){
```

```
    cin >> arr[i];
```

```
    sum += arr[i];
```

```
}
```

```
quicksort(arr, 0, N-1);
```

```
for (int i=0; i<N; i++){
```

```
    cout << arr[i] << " ";
```

```
    cout << endl;
```

```
    cout << "Average of TO S:" << sum/N << endl;
```

```
double overallAvg = (double)sum/N;
```

```
int count=0;
```

```
for (int i=0; i<N; i++){
```

```
    if (arr[i] > overallAvg){
```

```
        count++
```

```
    }
```

```
    cout << "Value Above overall Average:" << count << endl;
```

```
    return 0;
```

```
}
```